

Report

基础数据结构实现（包含 T1 Dijkstra）

1.1 图（邻接表）

图采用**邻接表**存储，选择理由：

- 地图为稀疏无向带权图，边数远小于 V^2 ，邻接表空间复杂度 $O(V + E)$ 优于邻接矩阵的 $O(V^2)$ 。
- Dijkstra 遍历邻边时只需访问实际存在的边，邻接表支持 $O(\deg(u))$ 遍历。

```
// graph.h
struct adj_edge { int u, v; double w; };

class graph {
    std::vector<node> nodes;
    std::vector<std::vector<adj_edge>> adj; // adj[u] = 所有邻边
    int n, m;
};
```

`add_edges(u, v, w)` 同时向 `adj[u]` 和 `adj[v]` 插入，维护无向性，时间复杂度 $O(1)$ 。

1.2 最小堆（手写，用于 Dijkstra）

使用**手写 1-indexed 二叉最小堆**，支持 `push`、`pop`、`decrease_key` 操作：

```
// min_heap.h
class MinHeap {
    std::vector<HeapNode> heap; // 1-indexed
    std::vector<int> pos; // pos[node_id] = 堆中下标, O(1) 定位
    void sift_up(int i);
    void sift_down(int i);
};
```

复杂度： `push` / `pop` / `decrease_key` 均为 $O(\log V)$ ，`contains` 为 $O(1)$ （通过 `pos` 数组）。

decrease_key 通过 pos 数组 $O(1)$ 定位节点，然后 sift_up 修复堆性质。Dijkstra 中每条边可能触发一次 decrease_key，故总复杂度 $O((V + E) \log V)$ 。

1.3 循环队列（手写）

固定容量的循环队列，用于维护驿站待配送任务队列：

```
// circular_queue.h
template <typename T>
class CircularQueue {
    std::vector<T> data;
    int head, tail, count, capacity;
};
```

push / pop / front 均为 $O(1)$ ，空间 $O(C)$ 。

1.4 T1 Dijkstra 最短路

实现为 dijkstra(const graph& g, int src)，返回 {dist[], prev[]}：

- dist[v] = 源点到 v 的最短距离
- prev[v] = 最短路径上 v 的前驱节点（可通过 get_path 回溯完整路径）

算法流程：

1. 初始化 dist[src]=0，其余 $+\infty$ ，源点入堆。
2. 每轮取出堆顶 u（当前最小 dist），松弛 u 的所有邻边 (u,v,w)：若 dist[u]+w < dist[v] 则更新并入堆/ decrease_key。
3. 堆空时结束。

时间复杂度： $O((V + E) \log V)$ （使用手写最小堆）。

正确性：标准 Dijkstra，所有边权非负，数学归纳法可证。

1.5 全源最短路

```
AllPairResult all_pairs_dijkstra(const graph& g);
```

对所有节点调用 dijkstra，得到 dist[i][j] 和 path[i][j]。后续 T2–T5 均基于此进行 $O(1)$ 距离查表。

时间复杂度： $O(V \cdot (E + V \log V))$ 。

T2 最小化不满意度和

2.1 问题描述

- 不考虑超时 ($T_i = +\infty$)，容量上限 w_{lim} 生效。
- 包裹 i 只有在其到站时间 S_i 之后才可配送。
- 目标：最小化 $\sum(D_i - S_i)$ 。

2.2 算法设计：贪心初解 + 模拟退火 (SA)

2.2.1 贪心初解

- 所有包裹按 S_i 升序排序（早到站的先安排）。
- 依次装入当前趟，重量超过容量则封趟、开新趟。
- 每趟内部用 **nearest-first** 确定目的地顺序：从驿站出发，每次贪心选择距离当前节点最近的目的地。

复杂度：排序 $O(n \log n)$ ，分趟 $O(n)$ ，趟内排序 $O(m^2)$ (m = 该趟目的地数)。

2.2.2 模拟退火优化

解表示： `trip_plans[t]` = 第 t 趟的包裹 id 列表， `dest_orders[t]` = 该趟的目的地访问顺序。

邻域操作（三类，等概率选取）：

op	描述	用途
0	跨趟搬单个包裹 (relocate)	调整包裹分配
1	趟内交换两个目的地 (swap)	优化访问顺序
2	趟内 2-opt (reverse 子路径)	消除交叉路径

评估函数： `evaluate_solution` 模拟全部趟，累计所有包裹的 $D_i - S_i$ 之和。

SA 框架：

- 接受准则 (Metropolis)： $\Delta < 0$ 无条件接受，否则以 $P = \exp(-\Delta/T)$ 接受。

- **温控**：Lundy-Mees 恒温退火， $T_{k+1} = T_k / (1 + \beta T_k)$ 。
- **自适应初始温度**：对初解做 100 次随机扰动，用上坡 Δ 的中位数反推 T_{init} ($\exp(-\Delta/T) = 0.8 \Rightarrow T = \Delta_{\text{med}}/0.223$)。
- **终止**：MAX_ITER = 100000 或 $T < T_{\text{min}} = 10^{-3}$ 。

时间复杂度：SA 每轮 $O(n)$ （评估），总计 $O(n \log n + \text{iter} \cdot n)$ 。

T3 带容量的运送成本

3.1 问题描述

- $S_i = 0$ ，包裹随时可配送。
- 有 deadline T_i ，超时包裹数作为输出指标。
- 成本定义：逐段累加"段长 × (自重 + 车上剩余包裹总重)"。
- 优化目标：优先压超时数，同超时下压总成本（用大 M 加权评分近似字典序）。

3.2 算法设计：EDD 贪心 + 多邻域 SA

3.2.1 贪心初解

1. 包裹按 deadline T_i **升序**（Earliest Deadline First, EDD）。
2. 依次贪心装车（容量 w_{lim} ），装满即封趟。
3. 趟内排序：**heaviest-first**——按目的地包裹总重降序，重者先送以降低后续路段成本。

同时生成 nearest-first 版本作为对照，两者评估后择优选为 SA 起点。

复杂度： $O(n \log n)$ 。

3.2.2 SA 优化（扩充邻域）

在 T2 的 3 种邻域操作基础上扩充至 **6 种**：

op	描述
0	跨趟搬单个包裹（relocate）
1	跨趟交换两个包裹（swap-between-trips）
2	跨趟搬运同一目的地的一组包裹（relocate-block）

op	描述
3	交换两趟的先后顺序 (swap-trips)
4	趟内交换两个目的地 (swap-in-trip)
5	趟内 2-opt (reverse sub-path)

评分函数： $\text{score} = M \cdot \text{overtime_count} + \text{total_cost}$, $M = 10^8$ 足够大使得超时数成为第一优先级。

SA 参数：

- $\text{MAX_ITER} = 150,000$
- $T_{\min} = 10^{-3}$
- 自适应 T_{init} (200 次扰动, 中位数反推)
- Lundy-Mees 恒温退火

复杂度：每轮评估 $O(n)$, 总 $O(\text{iter} \cdot n)$ 。

T4 退货回收 (TSP)

4.1 问题描述

- 容量视为无穷大 (无需分批)。
- 小车从驿站出发, 访问所有退货地点取回退货, 最终回到驿站。
- 目标：总耗时最短 (等价于总路径长度最短, 速度恒定)。

本质是**旅行商问题** (TSP)。

4.2 算法设计：遗传算法 (GA)

4.2.1 编码与适应度

- **个体：**退货节点的排列 (不含 0), 表示访问顺序 $[v_1, v_2, \dots, v_k]$ 。
- **回路长度：** $\text{len} = \text{dist}[0][v_1] + \sum \text{dist}[v_i][v_{i+1}] + \text{dist}[v_k][0]$ 。
- **适应度：** $f = 1/(\text{len} + \varepsilon)$, 越大越优。

4.2.2 遗传算子

- **交叉**：OX (Order Crossover)。在父代 a 中随机选一段复制给子代，其余位置按父代 b 的顺序填充，保证排列合法性。
- **变异** (复合)：
 - swap 变异 (30%)：随机交换两个位置
 - inversion 变异 (20%)：反转某段子序列
 - scramble 变异 (10%)：打乱某段子序列
- **选择**：锦标赛选择 (tournament size = 3)，选适应度最高的个体。

4.2.3 算法流程

1. 初始种群：80% 随机排列 + 20% 最近邻贪心 (保证初始解质量)。
2. 每代评估适应度，精英保留 top 2。
3. 锦标赛选择 + OX 交叉 ($p_c = 0.85$) + 复合变异 ($p_m = 0.30$) 生成下一代。
4. 连续 200 代无改进则提前终止。

参数：种群大小 = $\min(200, \max(20, 4n))$ ，最大代数 = $\max(500, \text{pop_size} \times 10)$ 。

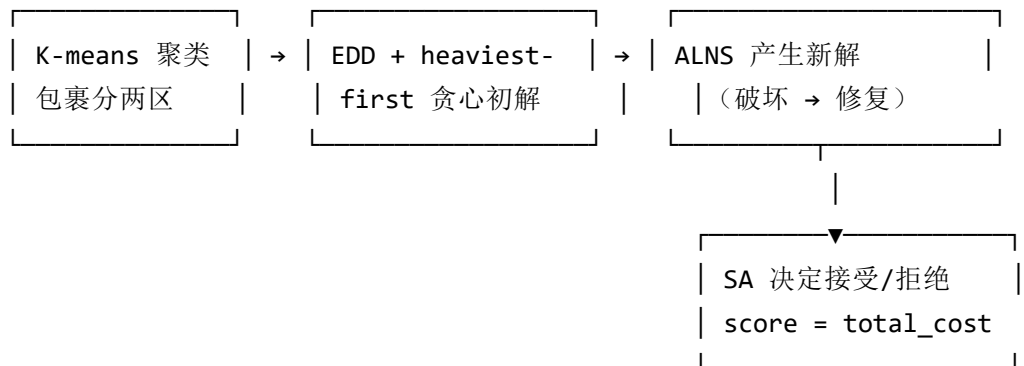
时间复杂度：每代 $O(\text{pop_size} \times n)$ ，总 $O(\text{gen} \times \text{pop_size} \times n)$ 。

T5 双车协同配送

5.1 问题描述

T5 在 T3 的基础上引入第二辆参数相同的小车。两车可同时从驿站 (0 号节点) 出发，各自装货与配送。成本定义同 T3——逐段累加“段长 $\times$ (自重 + 车上剩余包裹总重)”，**总成本为优化目标** (两车成本之和，越小越好)；超时包裹数仅作为统计项输出，不作为优化目标。这本质是一个 **2-CVRP (两车容量约束车辆路径问题)**，且附带 deadline 约束。

5.2 整体算法框架：K-means + ALNS + SA



1. **K-means 聚类分区**：将包裹按目的地空间位置聚为 2 类，各自分配给一辆车，得到一个地理上合理的初始分组。
2. **贪心初解**：每车内部使用与 T3 相同的 EDD（Earliest Deadline First）贪心分趟 + heaviest-first 趟内排序，生成可行方案。
3. **ALNS（Adaptive Large Neighborhood Search）**：每轮随机选取一个破坏算子和一个修复算子，对当前解进行较大范围的扰动，生成新解。
4. **SA（Simulated Annealing）**：以 `total_cost` 为目标函数逐轮判断是否接受新解，温控采用 Lundy-Mees 恒温退火。超时数仅作为统计输出，不参与 SA 评分。
5. **多起点重启**：进行 6 次独立求解（不同随机种子），取最优结果，缓解 K-means 初始分区的随机性影响。

5.3 K-means 初始分区

将所有包裹的目的地坐标（节点位置）作为特征，运行 K-means（ $k = 2$ ）得到两簇。每簇内的包裹分配给同一辆车。

- 若某车分配到的包裹总重超过 $w_{lim} \times \text{理论趟数}$ ，则将溢出包裹按 deadline 升序移到另一辆车（保证可行）。
- 若 K-means 完全聚为一簇（一车为空），回退为按 EDD 交替分配。
- 分区仅作为 SA 的“不算太差”的起点，后续 ALNS 的跨车操作可以纠正初始分配的不合理之处。

时间复杂度： $O(t \times n \times k) = O(t \times n)$ ，其中 t 为 K-means 迭代次数（ ≤ 100 ），远小于后续 SA 迭代次数。

5.4 贪心初始解

每车内部与 T3 完全一致：

- 1. 包裹按 deadline 升序排序 (EDD)。
- 2. 依次贪心装车，容量约束 w_lim 生效，装满即封趟。
- 3. 每趟内用 nearest-first 对目的地排序。

得到一个两车独立可行的 (trip_plans_car1, dest_orders_car1) 和 (trip_plans_car2, dest_orders_car2)。

5.5 ALNS 算子设计

每轮 SA 迭代中，随机（按自适应权重）选择一个破坏算子 + 一个修复算子。

破坏算子 (Destroy)

每次破坏移除约 20% 的包裹，放入"待插入池"。为控制计算开销，**所有破坏算子均使用廉价代理指标评分**，不跑完整 evaluate_solution。

算子	做法	复杂度
Worst-removal (代理版)	对每个包裹用 $dist[0][dest_i] \times weight_i$ 打分（远距离重包裹代价高），取分数最高的 k 个移除。	$O(n \log n)$
Shaw-removal	随机选一个包裹种子，移除与它"相关度最高"的 k-1 个包裹（相关度 = $\gamma_1 \times \text{地理距离} + \gamma_2 \times \text{deadline 差异} + \gamma_3 \times \text{重量差异}$ ，归一化后加权求和， $\gamma = [0.5, 0.3, 0.2]$ ）。	$O(n^2)$
Trip-destroy	随机选择一辆车的一整趟，全部包裹放入待插入池。	$O(1)$
Time-window-removal	移除所有当前超时的包裹 + 随机移除若干与其同趟的包裹（约 30%）。	$O(n)$

修复算子 (Repair)

将待插入池中的包裹逐个重新插入（支持跨车插入），同样使用局部代理指标，不跑完整评估。

算子	做法	复杂度
Greedy-cheapest (代理版)	对每个待插入包裹，遍历两车所有趟的所有可能插入位置，计算 局部边权增量 $\Delta = dist[prev][dest_i] + dist[dest_i][next] - dist[prev][next]$ ($O(1)$ 查表)。选择 Δ 最小的位置插入。 引入负载均衡惩罚项，鼓励往总重较轻的车插入。 若插入后超出容量，对该车新增一趟。	$O(q \times m \times l)$

算子	做法	复杂度
Regret-2	对每个待插入包裹，计算最佳位置与次佳位置的 Δ 之差（regret 值），优先插入 regret 最大的包裹。这样可避免"先插入的占便宜，后插入的没地方放"。	$O(q \times m \times l)$

其中 q = 待插入包裹数（约 $0.2n$ ）， m = 总趟数， l = 平均每趟目的地数。

关键设计原则

- 破坏/修复只用代理指标：完整 `evaluate_solution` 仅在每个新解生成后跑一次（用于 SA 接受判断）。
- 跨车操作内置在修复算子中：修复时同时考虑两辆车，自然纠正 K-means 初始分区的误差。
- 自适应权重：各破坏/修复算子初始权重相等，每轮 SA 后若降低了 `best_cost`，则该轮使用的算子在轮盘赌中权重+1，使有效算子被更频繁选中。

5.6 SA 接受准则与温控

- 评分函数：`score = total_cost`（两车总运送成本之和），超时不参与评分。
- 接受准则（Metropolis）：
 - 若 $\Delta cost < 0$ ：无条件接受。
 - 若 $\Delta cost \geq 0$ ：以概率 $\exp(-\Delta cost/T)$ 接受。
 - 若成本平手且超时减少：接受。
- 温控：Lundy-Mees 恒温退火， $T_{k+1} = T_k / (1 + \beta \cdot T_k)$ 。
- 自适应初始温度：对初解做 100 次随机扰动，用上坡 $\Delta cost$ 的中位数反推 T_{init} ，使初始接受率约为 0.8。
- 终止：`MAX_ITER = 20000`，或 $T < T_{min} = 10^{-3}$ 。
- 多起点重启：进行 6 次独立求解（不同种子），取最优。

5.7 整体时间复杂度

设 n = 包裹数， E = 图边数，`iter` = SA 迭代次数（含 6 次重启）。

阶段	复杂度
全源最短路（Dijkstra $\times V$ ）	$O(V \times (E + V \log V))$
K-means 分区	$O(t \times n)$
贪心初解	$O(n \log n)$

阶段	复杂度
SA 每轮	
— 破坏算子	$O(n \log n)$ 或 $O(n^2)$ (Shaw-removal)
— 修复算子	$O(q \times m \times l) \approx O(n)$
— 完整评估 (1 次)	$O(n \times \text{平均趟数})$
总计	$O(V \cdot (E + V \log V) + \text{iter} \cdot n \log n)$

与 T3 的 SA 处在同一量级，常数因子约 2–3 倍（两车评估开销翻倍 + ALNS 额外算子开销）。

5.8 两车时间线与最终输出

- 两车独立各自从 $t = 0$ 出发，时间线互不干扰。
- 优化目标：**总成本 = 车 1 总成本 + 车 2 总成本，越小越好。
- 统计项：**总超时 = 车 1 超时包裹数 + 车 2 超时包裹数，不参与优化。
- 最终输出两车各自的 trip 列表、总成本、超时数。

总结

任务	核心算法	优化方法	关键复杂度
T1	Dijkstra + 手写最小堆	—	$O((V + E) \log V)$
T2	EDD 贪心 + 最近邻趟内排序	SA (3 邻域 + Lundy-Mees 温控)	$O(\text{iter} \cdot n)$
T3	EDD 贪心 + heaviest-first 趟内排序	SA (6 邻域 + 大 M 加权评分)	$O(\text{iter} \cdot n)$
T4	遗传算法	OX 交叉 + 复合变异 + 锦标赛选择	$O(\text{gen} \cdot \text{pop} \cdot n)$
T5	K-means 分区 + EDD 贪心	ALNS (4 破坏 + 2 修复) + SA (6 重启)	$O(\text{iter} \cdot n \log n)$